

	UNIVERSIDAD NACIONAL DE SAN AGUSTIN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMA	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 1

INFORME DE LABORATORIO

INFORMACIÓN BÁSICA					
ASIGNATURA:	TECNOLOGÍA DE OBJETOS				
TÍTULO DE LA PRÁCTICA:	INDUCCIÓN A SCALA				
NÚMERO DE PRÁCTICA:	01	AÑO LECTIVO:	2026-B	NRO. SEMESTRE:	VI
FECHA DE PRESENTACIÓN	06/09/2026	HORA DE PRESENTACIÓN	11:59 p.m.		
INTEGRANTE (s): - Mejia Rondan Giovanni Patrick - Cornejo Hurtado Dario Rafael - Palma Apaza Santiago Enrique				NOTA:	
DOCENTE(s): Ing. Edith Giovanna Cano Mamani					

SOLUCIÓN Y RESULTADOS
I. SOLUCIÓN DE EJERCICIOS/PROBLEMAS 1. DESARROLLO 1.1. Visite el sitio oficial de Scala u otros para estudiar el lenguaje de programación Scala. 1.2. Estudie el problema recursivo propuesto. 1.3. Resuelva el problema con el lenguaje de programación Scala (Recursivamente). 1.4. Responda las tres preguntas formuladas. 2. PROBLEMA PROPUESTO <u>Vuelto - Contando el vuelto o cambio</u> Escriba una función recursiva que cuente de cuántas maneras diferentes puede dar cambio para una determinada cantidad y de acuerdo a una lista de denominaciones de monedas. Por ejemplo, hay 3 maneras de dar cambio si la cantidad=4 si tienes monedas con denominación 1 y 2: • 1+1+1+1, • 1+1+2,

	UNIVERSIDAD NACIONAL DE SAN AGUSTIN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMA	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 2

● **2+2.**

Realiza este ejercicio implementando la función countChange en Scala.

Esta función toma una cantidad a cambiar y una lista de denominaciones únicas para las monedas.

Su definición es la siguiente:

def countChange(money: Int, coins: List[Int]): Int

Puedes usar las funciones isEmpty, head y tail en la lista de monedas enteras.

3. Resolución de la práctica

3.1. Exploración dentro del sitio oficial de Scala

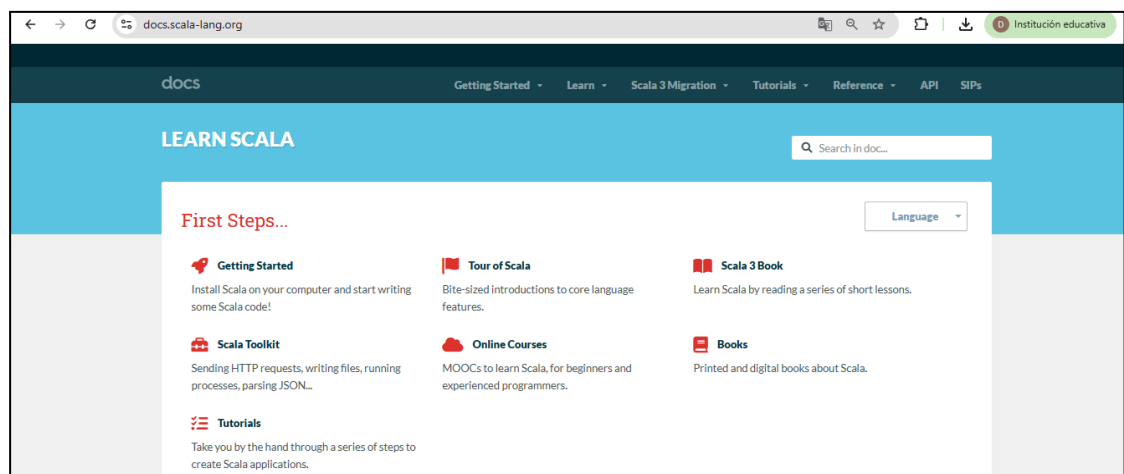


Figura 1. Se accedió al sitio oficial de documentación de Scala, sección "Learn Scala", donde se listan los principales recursos de aprendizaje del lenguaje: Getting Started, Tour of Scala, Scala 3 Book, Scala Toolkit, Online Courses, Tutorials y Books. Esta página sirvió como punto de partida para explorar el lenguaje antes de resolver el problema recursivo propuesto.

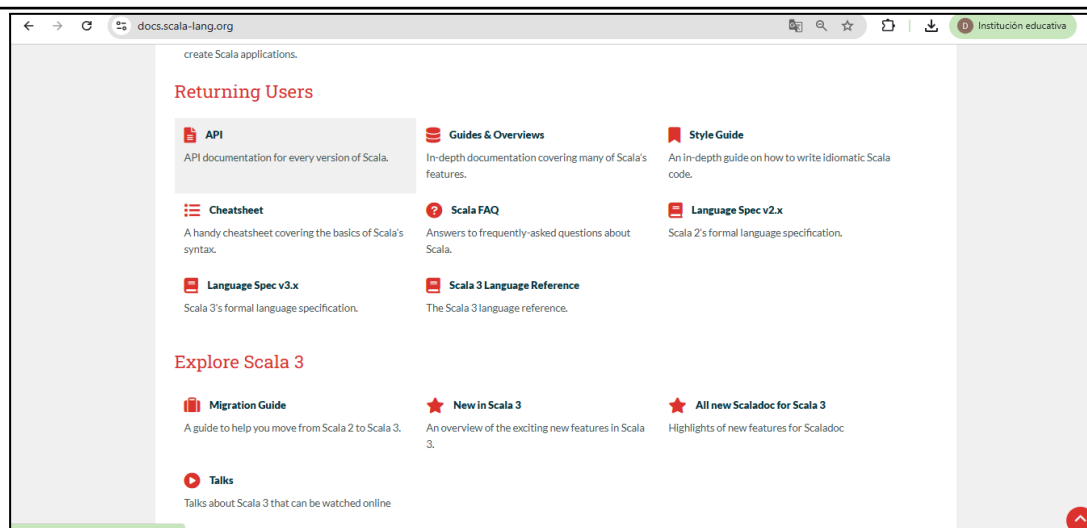


Figura 2. Se exploró la sección "Returning Users", que ofrece recursos avanzados como la documentación de la API, guías (Guides & Overviews), Style Guide, Cheatsheet, FAQ y las especificaciones formales del lenguaje (Language Spec v2.x y v3.x). Asimismo, en "Explore Scala 3" se identificaron recursos sobre la migración de Scala 2 a Scala 3 (Migration Guide), las nuevas características de esta versión (New in Scala 3) y charlas relacionadas (Talks).

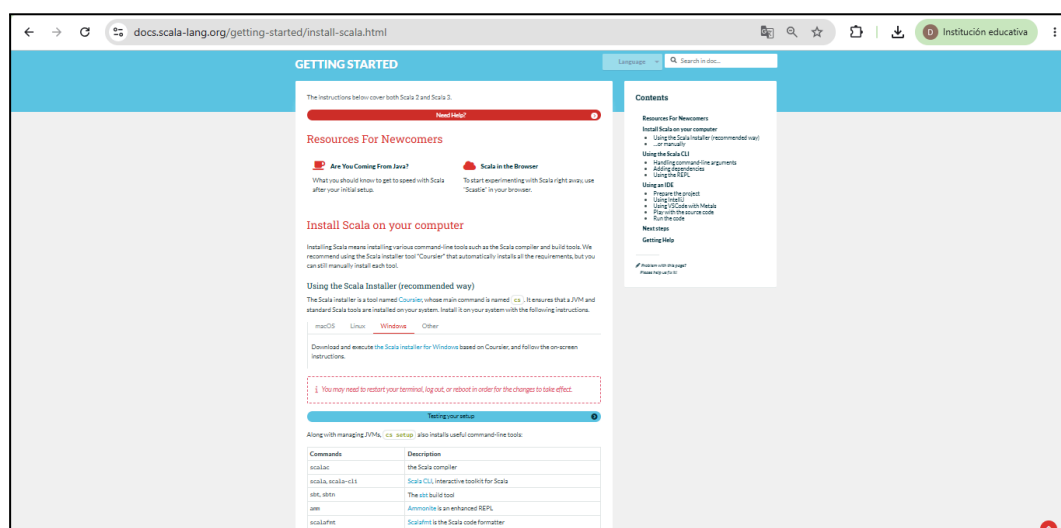


Figura3. Se revisó la guía oficial de instalación de Scala, la cual recomienda el uso del instalador "Coursier" (comando cs) para instalar automáticamente la JVM y las herramientas estándar del lenguaje. La página detalla además los principales comandos disponibles tras la instalación: scalac (compilador), scala/scala-cli (toolkit

interactivo), sbt (herramienta de build), amm (REPL mejorado) y scalafmt (formateador de código).

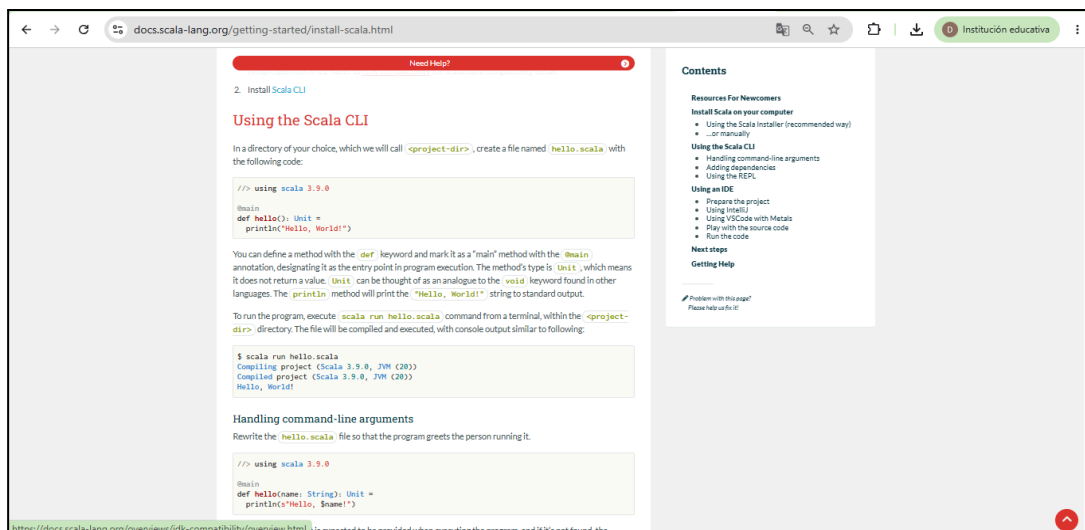


Figura 4. Se estudió el ejemplo básico de "Hello, World!" en Scala mediante la Scala CLI, donde se define un método hello() con la anotación @main como punto de entrada del programa. Se explicó el uso del tipo Unit (análogo a void) y la ejecución del programa con el comando scala run hello.scala desde la terminal. También se revisó cómo extender el ejemplo para recibir argumentos por línea de comandos (parámetro name: String).

3.2. Estudio previo del problema propuesto

El problema plantea determinar de cuántas formas distintas se puede obtener una cantidad de dinero (money) a partir de una lista de denominaciones de monedas disponibles (coins), sin importar el orden en que se combinen las monedas (por ejemplo, 1+1+2 se considera igual a 1+2+1)

Análisis de la estructura recursiva:

Para resolver este problema se identifica que el conteo total de combinaciones puede descomponerse en dos subproblemas más pequeños, dependiendo de si se usa o no la primera denominación de la lista:

- No usar la primera moneda de la lista: se cuentan las formas de dar el cambio usando únicamente el resto de denominaciones (coins.tail).

- Usar al menos una vez la primera moneda de la lista: se cuenta el número de formas de dar cambio para la cantidad restante (money - coins.head), permitiendo seguir usando esa misma moneda (por eso la lista de denominaciones no se reduce en este caso).

La suma de ambos casos da el total de combinaciones posibles.

Casos base identificados:

- Si money == 0: existe exactamente 1 forma de dar el cambio (no usar ninguna moneda), por lo que se retorna 1.
- Si money < 0: la combinación no es válida (nos pasamos del monto), se retorna 0.
- Si money > 0 y la lista de monedas está vacía (coins.isEmpty): no hay forma de completar el cambio, se retorna 0.

Relación de recurrencia:

None

```
countChange(money, coins) =
    1                                si money == 0
    0                                si money < 0
    0                                si money > 0 y
coins.isEmpty
    countChange(money, coins.tail) + countChange(money - coins.head, coins)
en otro caso
```

Este análisis evidencia que el problema tiene subestructura óptima (se resuelve combinando soluciones de subproblemas más pequeños) y es naturalmente expresable con recursión usando las funciones isEmpty, head y tail sobre la lista de monedas, tal como sugiere el enunciado.

3.3. Resolución completa de problema propuesto

Con base en el análisis previo, se implementó la función recursiva countChange en el archivo vuelto.scala, editado mediante el editor nano desde el entorno WSL (Windows Subsystem for Linux), mapeando la ruta del proyecto en /mnt/c/Users/Usuario/Documents/TO-Laboratorio/Lab 01. El código desarrollado fue el siguiente:

	UNIVERSIDAD NACIONAL DE SAN AGUSTIN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMA	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 6

```

File: vuelto.scala
1  def countChange(money: Int, coins: List[Int]): Int = {
2      if (money == 0) 1
3      else if (money < 0) 0
4      else if (coins.isEmpty) 0
5      else countChange(money, coins.tail) + countChange(money - coins.head, coins)
6  }
7
8  @main def main(): Unit = {
9      val monto = 4
10     val monedas = List(1, 2)
11     println(s"Formas de dar cambio para $monto con $monedas: ${countChange(monto, monedas)}")
12 }

lario@DESKTOP-HQFP4Q3:/mnt/c/Users/Usuario/Documents/TO-Laboratorio/Lab 01$

```

La función `countChange` recibe la cantidad de dinero (`money`) y la lista de denominaciones (`coins`), y evalúa recursivamente los casos base (`money == 0`, `money < 0`, `coins.isEmpty`) descritos en el análisis previo, sumando las combinaciones que resultan de no usar la primera denominación (`coins.tail`) y de usarla al menos una vez (`money - coins.head, coins`).

Dado que la instalación de Scala se realizó desde la terminal de Windows (CMD) y no dentro de WSL, la ejecución del programa se llevó a cabo desde el CMD.

Salida en consola

```

C:\WINDOWS\system32\cmd. x + v

C:\Users\Usuario\Documents\TO-Laboratorio\Lab 01>cd C:\Users\Usuario\Documents\TO-Laboratorio\Lab 01>
La sintaxis del comando no es correcta.

C:\Users\Usuario\Documents\TO-Laboratorio\Lab 01>scala run vuelto.scala

Downloading JVM temurin:17r/jvm-index/raw/master/index.json
Downloading compilation server 2.1.17-binaries/releases/download/jdk-17.0.20.1%2B1/OpenJDK17U-jdk_x64_windows_hotspo...
Starting compilation serveren2/org/scala-sbt/ivy/ivy/2.3.0-sbt-77cc781d727b367d3761f097d89f5a4762771d41/ivy-2.3.0-sb...
Compiling project (Scala 3.9.0, JVM (17))s
Compiled project (Scala 3.9.0, JVM (17))
Formas de dar cambio para 4 con List(1, 2): 3

C:\Users\Usuario\Documents\TO-Laboratorio\Lab 01>

```

El resultado obtenido (3 formas) coincide exactamente con el ejemplo propuesto en el enunciado del problema (1+1+1+1, 1+1+2, 2+2), validando así la correcta implementación de la función recursiva `countChange`.⁷

4. Entregables finales:

Enlace de github: <https://github.com/GiovanniDev06/TO-Laboratorio>

	UNIVERSIDAD NACIONAL DE SAN AGUSTIN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMA	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 7

Scala code runner version: 1.16.0

Scala version (default): 3.9.0

II. SOLUCIÓN DEL CUESTIONARIO

1. Explique el caso base implementado.

La función countChange implementa tres casos base, que constituyen las condiciones de parada de la recursión:

- $money == 0 \rightarrow$ retorna 1: cuando la cantidad restante por cubrir llega exactamente a cero, significa que se encontró una combinación válida de monedas que suma el monto total solicitado. Por eso se cuenta como una forma válida de dar el cambio.
- $money < 0 \rightarrow$ retorna 0: ocurre cuando, al restar el valor de una moneda, el monto restante se vuelve negativo. Esto indica que la combinación de monedas utilizada excedió la cantidad a cambiar, por lo que esa rama de la recursión es inválida y no aporta ninguna solución.
- `coins.isEmpty` (con $money > 0$) $\rightarrow$ retorna 0: sucede cuando ya no quedan denominaciones disponibles para seguir probando combinaciones, pero aún resta una cantidad de dinero por cubrir. Al no existir más monedas con qué completar el monto, esa rama tampoco produce ninguna solución válida.

Estos tres casos base son los que detienen el árbol de llamadas recursivas y determinan si una rama específica de combinaciones contribuye (+1) o no (+0) al conteo total de formas de dar el cambio.

2. ¿Este problema se resuelve mejor en el modelo de programación funcional?

Sí, el problema del conteo de vuelto se adapta de forma natural y elegante al modelo de programación funcional, principalmente por las siguientes razones:

- **Recursión como mecanismo central:** el problema tiene una estructura recursiva evidente (subestructura óptima), donde la solución se construye combinando soluciones de subproblemas más pequeños (`countChange(money, coins.tail)` y `countChange(money - coins.head, coins)`). Este estilo de descomposición encaja directamente con el paradigma

	UNIVERSIDAD NACIONAL DE SAN AGUSTIN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMA	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 8

funcional, que privilegia la recursión sobre las estructuras de control iterativas (bucles) propias de la programación imperativa.

- **Inmutabilidad y funciones puras:** la función `countChange` no modifica ningún estado externo ni depende de variables mutables; dado el mismo `money` y la misma lista `coins`, siempre retorna el mismo resultado. Esto la convierte en una función pura, característica esencial del paradigma funcional que facilita el razonamiento sobre el código y evita efectos colaterales.
- **Uso de listas inmutables y sus operadores:** el problema se apoya en operaciones típicas de listas funcionales (`isEmpty`, `head`, `tail`) que descomponen la estructura de datos sin modificarla, en lugar de recorrerla con índices y variables mutables como se haría en un enfoque imperativo con arreglos.
- **Claridad y concisión:** la solución funcional resulta más compacta y cercana a la definición matemática/recurrente del problema, en comparación con una versión imperativa que normalmente requeriría una tabla de programación dinámica y bucles anidados para lograr el mismo resultado.

3. ¿Qué beneficios se podrían obtener para este problema utilizando memoización?

La implementación recursiva actual de `countChange` presenta subproblemas superpuestos (overlapping subproblems): distintas ramas del árbol de recursión terminan calculando el mismo par (`money`, `coins`) en más de una ocasión, lo que provoca una cantidad de llamadas que crece de forma exponencial respecto al tamaño de la entrada.

Aplicar memoización —almacenando en una estructura auxiliar (por ejemplo, un `Map[(Int, List[Int]), Int]`) el resultado de cada combinación (`money`, `coins`) ya calculada— traería los siguientes beneficios:

- **Reducción drástica de la complejidad temporal:** al evitar recalcular subproblemas ya resueltos, la complejidad pasa de exponencial a polinomial, mejorando notablemente el rendimiento para montos y listas de monedas grandes.
- **Mejor escalabilidad:** el algoritmo memoizado puede manejar cantidades (`money`) y conjuntos de denominaciones más amplios sin que el tiempo de ejecución se dispare.
- **Se conserva el estilo funcional:** la memoización puede implementarse manteniendo la naturaleza recursiva y declarativa de la solución, sin necesidad de reescribir el algoritmo en un estilo completamente imperativo.
- **Trazabilidad de resultados intermedios:** al guardar los resultados de subproblemas, es posible reutilizar esos valores para resolver problemas relacionados, evitando recomputación innecesaria.

En síntesis, la memoización no cambia la lógica ni la corrección del algoritmo, pero optimiza significativamente su eficiencia al eliminar el recálculo redundante de los mismos subproblemas.

	UNIVERSIDAD NACIONAL DE SAN AGUSTIN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMA	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 9

III. CONCLUSIONES

- IV. **Facilidad de Scala para la recursión:** El lenguaje permitió pasar la idea del problema matemático directo al código sin tantas vueltas [2]. Al usar listas normales y operaciones sencillas como isEmpty, head y tail, no hizo falta crear bucles for ni variables que vayan cambiando de valor, dejando un código corto y fácil de entender [2].
- V. **Importancia de definir bien las paradas:** Para que el programa no se quede en un bucle infinito, fue clave definir los tres casos de corte [1]. Separar cuándo llegamos exacto al monto (money == 0), cuándo nos pasamos (money < 0) y cuándo nos quedamos sin monedas (coins.isEmpty), hizo que el programa se detenga a tiempo y bote exactamente las 3 combinaciones que pedía el enunciado [1].
- VI. **Ventajas y límites de la solución:** Aunque resolverlo con recursión directa es rápido de programar y queda limpio, el algoritmo repite varias veces las mismas operaciones cuando los números crecen [1], [3]. Para montos más grandes convendría guardar las respuestas que ya se calcularon (memoización) para que el programa no se vuelva lento ni sature la memoria [3].
- VII. **Uso de Scala 3 y la terminal:** Probar Scala 3 con herramientas como scala-cli facilitó bastante el trabajo [4]. Con la etiqueta @main pudimos correr el archivo vuelto.scala directo desde la consola sin tener que configurar proyectos pesados o crear estructuras complicadas de carpetas [2], [4].

RETROALIMENTACIÓN GENERAL

- **Organizar mejor el entorno de trabajo:** Se editó el código con nano en la terminal de WSL (Linux) y luego se ejecutó en el CMD de Windows. Para los siguientes laboratorios conviene tener todo instalado y corriendo en un solo lado (preferentemente dentro de Linux/WSL) para no cruzar rutas de archivos ni tener errores raros con las barras invertidas de Windows.
- **Probar casos más pesados:** El ejercicio se probó con valores chicos y funcionó a la primera. Estaría bueno en futuras prácticas meterle montos más grandes para ver en qué punto la recursión empieza a sufrir y probar ahí mismo la solución guardando resultados en memoria para notar la diferencia de velocidad.

REFERENCIAS Y BIBLIOGRAFÍA

	UNIVERSIDAD NACIONAL DE SAN AGUSTIN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMA	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 10

- [1] H. Abelson, G. J. Sussman, y J. Sussman, *Structure and Interpretation of Computer Programs*, 2.ª ed. Cambridge, MA, USA: MIT Press, 1996, cap. 1, sec. 1.2.2, pp. 40–44.
- [2] M. Odersky, S. Spoon, y B. Venners, *Programming in Scala*, 5.ª ed. Walnut Creek, CA, USA: Artima Press, 2021, cap. 16 y cap. 22.
- [3] T. H. Cormen, C. E. Leiserson, R. L. Rivest, y C. Stein, *Introduction to Algorithms*, 4.ª ed. Cambridge, MA, USA: MIT Press, 2022, cap. 14, pp. 384–400.
- [4] Scala Documentation, «Scala 3 Book: Functional Programming», *Scala Documentation*, 2024. [En línea]. Disponible en: <https://docs.scala-lang.org/scala3/book/fp-intro.html>